EECS 482
Introduction to operating systems

Page replacement and eviction

Peter Chen
University of Michigan

Translation lookaside buffer (TLB)

- TLB: a cache for page table entries
  - TLB hit → Skip all the translation steps
  - TLB miss → Get PTE, store in TLB, restart instruction
    - » x86, ARM: done by hardware
    - » MIPS: done by software

- Must invalidate TLB on context switch, or identify entries with address space ID

Pro tip: indirection + caching

- Indirection: solves all problems in CS!
  - Except performance ☹
- Caching: solves the performance problems caused by indirection
  - Pointers added by indirection are small, so are cached easily

Caching and virtual memory

- CPU caches are a cache for …?
- The TLB is a cache for …?
- Physical memory is a cache for ...?

- How do the concepts from CPU caches (EECS 370) relate to the concepts of virtual memory?
  - CPU caches: direct mapped, set associativity, tags
  - VM: virtual and physical addresses, page tables
  - Are these related?

CPU caching and virtual memory

- What placement algorithm (associativity) did you use in EECS 370?
- What associativity is used in VM?
  - Why?

CPU caching and virtual memory

- In EECS 370, how did you find a page in the set?
- Will this work for virtual memory?
- How would you solve this in EECS 281?

Page replacement

- Which page to evict when you need a free physical page?
  - Goal: minimize page faults

FIFO

- FIFO
  - Replace page brought into memory longest time ago
  - May replace pages that continue to be frequently used

time →

Access trace	A	B	C	D	C	E	A	C	D	B	C
Resident pages	A	B	C	D	D	E	A	A	A	B	C
		A	B	C	C	D	E	E	E	A	B
			A	B	B	C	D	D	D	E	A
				A	A	B	C	C	C	D	E

Optimal replacement (OPT)

- Replace page that won't be used for the longest time in the future
  - Minimizes cache misses
  - Problems?

Access trace	A	B	C	D	C	E	A	C	D	B	C
Resident pages	A	A	A	A	A	A	A	A	A	A	A
		B	B	B	B	E	E	E	E	B	B
			C	C	C	C	C	C	C	C	C
				D	D	D	D	D	D	D	D

Least recently used (LRU)

- Replace the page that was last used longest ago
  - If page hasn't been used for a while, it probably won't be used for a long time in the future
  - Approximates OPT because of temporal locality: the future tends to reflect the past
  - Problems?

Access trace	A	B	C	D	C	E	A	C	D	B	C
Resident pages	A	B	C	D	C	E	A	C	D	B	C
		A	B	C	D	C	E	A	C	D	B
			A	B	B	D	C	E	A	C	D
				A	A	B	D	D	E	A	A

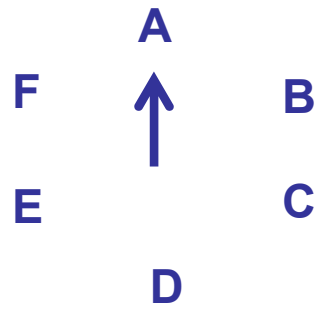
Approximating LRU

- Most MMUs maintain a “referenced” bit for each resident page
  - Set by MMU when page is read or written
  - Can be cleared by OS

- How to use reference bit to identify pages that haven’t been used for a while?

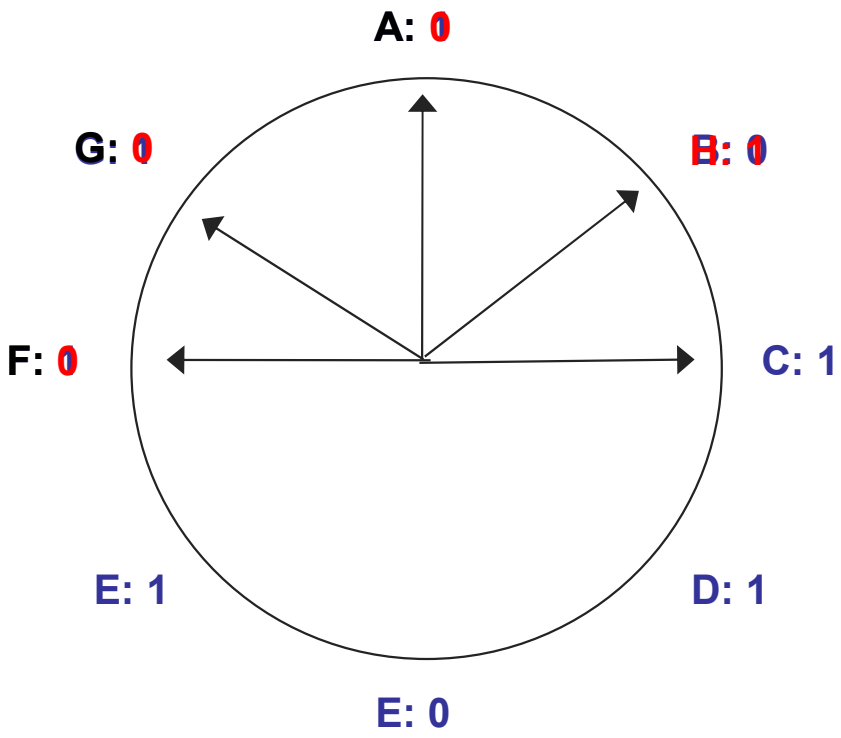
Clock replacement algorithm

- Arrange resident pages around a clock



- To choose which page to evict:
  - Consider page pointed to by clock hand
  - If referenced bit is set, it has been referenced “recently”
    - » Clear reference bit and advance clock hand
  - If referenced bit is not set, it has not been referenced “recently”
    - » Evict page

Clock example



- What if all pages referenced since last sweep?

Page eviction

- What to do with data from evicted page? Why?
- When do you not need to write page to disk?

- This is a “write-back” cache

- To tell if page has been modified since being read from disk, most MMUs provide a “dirty” bit for each resident page. MMU sets “dirty” bit when page is written.
  - “dirty” means “different from disk”

- Why not write to disk on every store (write-through cache)?

Optimizing when to do work

do work earlier: may reduce waiting later

do work later: may reduce total work

time →

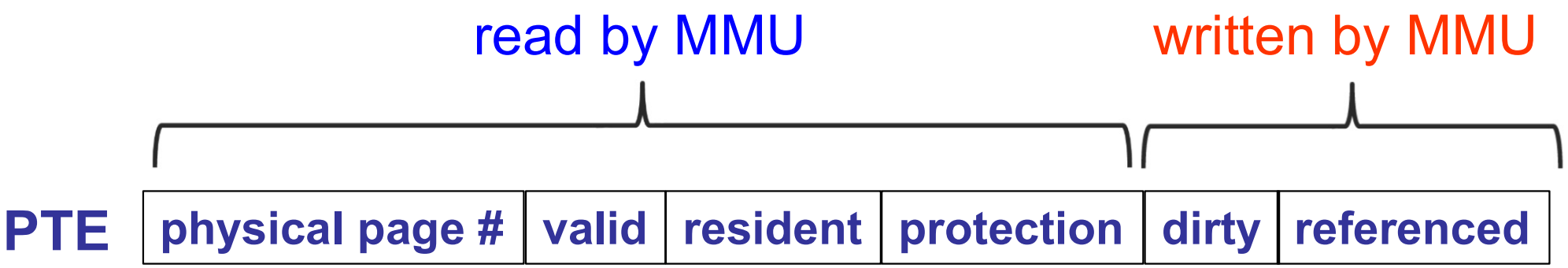
- When would you not need to write a new data value to disk?
- Project 3 memory manager should be as lazy as possible (with CLOCK replacement)

Midterm

- Monday, October 20, 5:00-6:50 pm

- Types of questions
  - Analysis
  - Synthesis
- Topics
  - OS history
  - Processes
  - Concurrent programming
  - Implementing concurrency

MMU algorithm




```
if (virtual page is invalid or non-resident or protected) {
        trap to OS fault handler
        retry access
} else {
        physical page # = pageTable[virtual page #].physPageNum
        pageTable[virtual page #].referenced = true
        if (access is write) {
                pageTable[virtual page #].dirty = true
        }
        access physical memory at {physical page #}{offset}
}
```

Maintaining information for virtual pages

Page table

vpage	ppage	prot	ref	dirty	resident	valid
7						
6		!r,!w				
5		?r,?w				
4						
3						
2						
1						
0						

OS page info

		disk block
	0	
0	1	

```
if (virtual page is invalid or non-resident or protected)
        trap to OS fault handler
```